VISUALISATION LIBRARY DOCUMENTATION

1. Introduction

Data visualization is the process of representing data using graphs, charts, and other visual elements. It helps users understand complex information more easily by identifying patterns, trends, and relationships within the data. Instead of reading large tables of numbers, visualizations make it easier to analyze and interpret information quickly.

Python is one of the most popular programming languages for data analysis and visualization because it provides powerful libraries for creating different types of charts and graphs. Visualization libraries such as **Matplotlib** and **Seaborn** allow users to present data in a clear, attractive, and meaningful way. These libraries are widely used in business, education, scientific research, and machine learning.

This documentation provides an overview of Matplotlib and Seaborn, these libraries are demonstrated using the Delhi Air Quality (AQI) dataset, which contains measurements of pollutants such as Carbon Monoxide (CO), Nitric Oxide (NO), Nitrogen Dioxide (NO_2), Ozone (O_3), Sulfur Dioxide (SO_2), PM2.5, PM10, and Ammonia (NH_3). The objective is to understand how different visualization techniques help analyze air pollution data effectively.

2. Matplotlib

2.1 Overview

Matplotlib is one of the most widely used Python libraries for data visualization. It was developed by John D. Hunter and is designed to create high-quality graphs, charts, and plots. It provides a simple and flexible interface for visualizing data and is suitable for both beginners and advanced users. It is widely used in data science, engineering, research, and business analytics.

Main Features

- Creates a wide variety of charts and graphs.
- Highly customizable with colors, labels, titles, and styles.
- Works well with libraries like NumPy and Pandas.
- Can generate high-quality figures for reports and presentations.
- Supports both 2D and basic 3D visualizations.

Common Uses

- Data analysis and reporting
- Scientific and engineering research
- Business analytics
- Academic projects
- Machine learning visualization

2.2 Graph Types

2.2.1 Line Plot

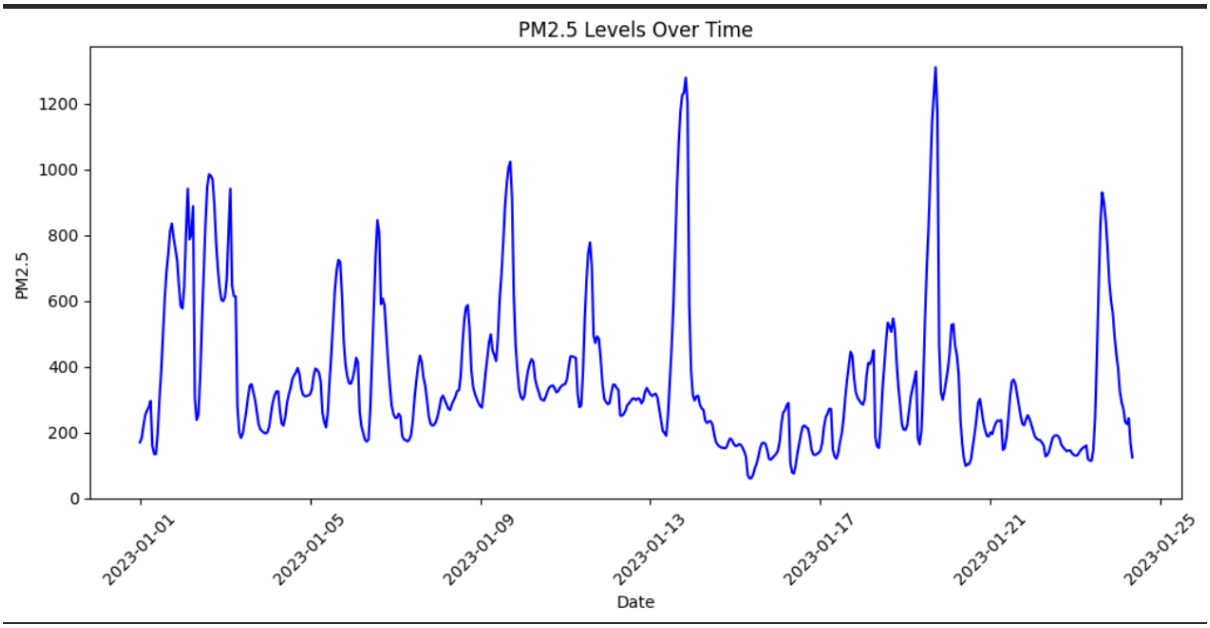
Description

A line plot is used to display data points connected by straight lines. It is mainly used to show trends or changes over time.

Use Case

A line plot is useful for visualizing monthly sales, temperature changes, stock prices, or any data that changes continuously over time.

```
import matplotlib.pyplot as plt

plt.figure(figsize=(12,5))

plt.plot(df["date"], df["pm2_5"], color="blue")

plt.title("PM2.5 Levels Over Time")
plt.xlabel("Date")
plt.ylabel("PM2.5")

plt.xticks(rotation=45)

plt.show()
```



2.2.2 Scatter Plot

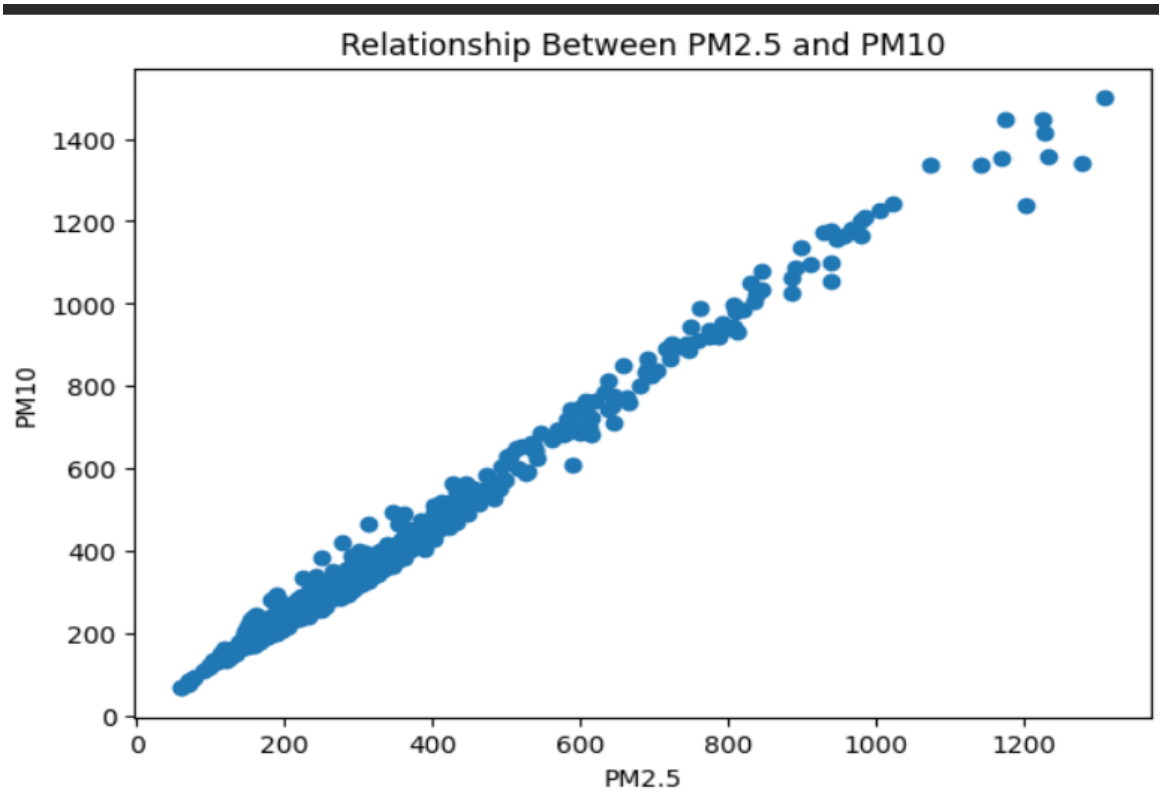
Description

A scatter plot is used to display the relationship between two numerical variables. Each point on the graph represents one observation. It helps identify patterns, trends, and outliers in the data.

Use Case

A scatter plot is useful for analyzing the relationship between variables, such as the number of study hours and exam scores, or advertising expenditure and sales

```
plt.figure(figsize=(7,5))

plt.scatter(df["pm2_5"], df["pm10"])

plt.title("Relationship Between PM2.5 and PM10")
plt.xlabel("PM2.5")
plt.ylabel("PM10")

plt.show()
```



2.2.3 Bar Chart

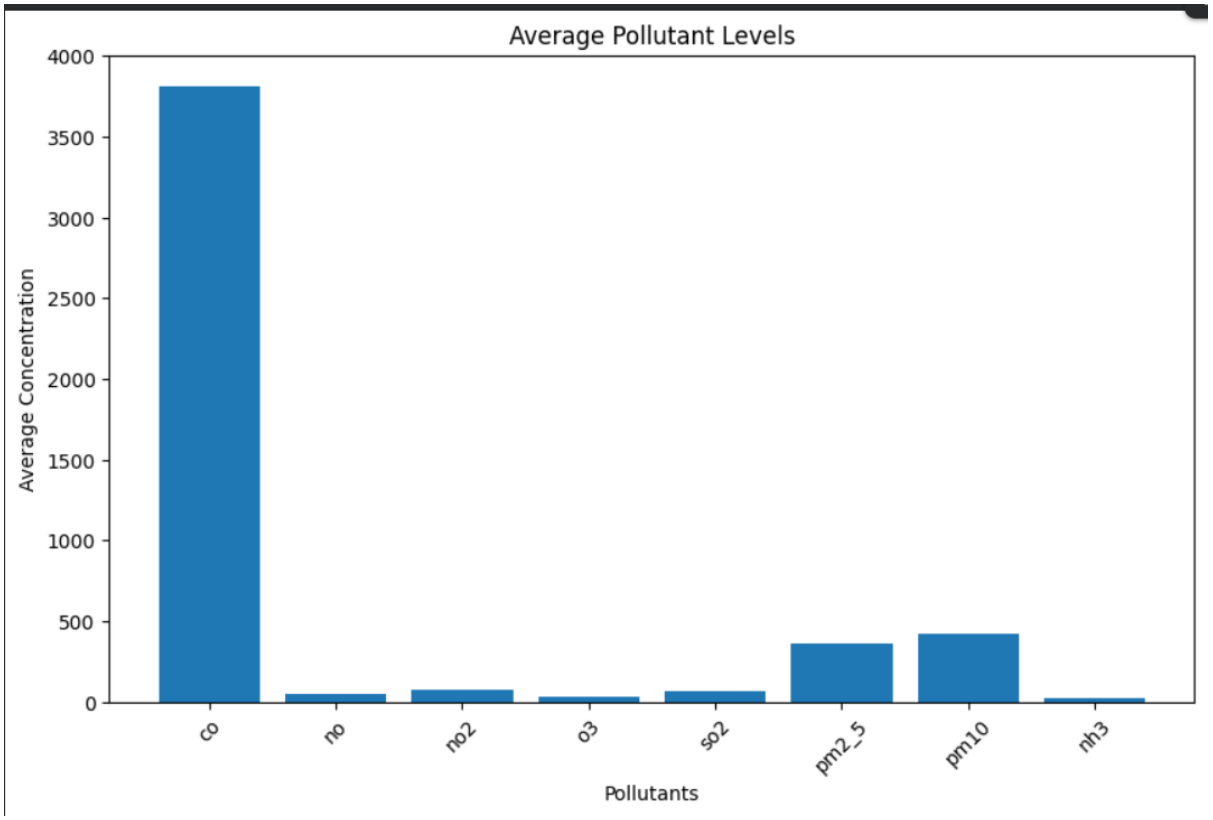
Description

A bar chart is used to compare values across different categories. Each category is represented by a rectangular bar, and the height of the bar indicates its value.

Use Case

A bar chart is useful for comparing sales of different products, the number of students in different classes, or the population of different cities.

```
average = df[["co","no","no2","o3","so2","pm2_5","pm10","nh3"]].mean()

plt.figure(figsize=(10,6))

plt.bar(average.index, average.values)

plt.title("Average Pollutant Levels")
plt.xlabel("Pollutants")
plt.ylabel("Average Concentration")

plt.xticks(rotation=45)

plt.show()
```


Average Pollutant Levels
Average Concentration
4000
3500
3000
2500
2000
1500
1000
500
0
co
no
no2
o3
so2
pm2_5
pm10
nh3
Pollutants

2.2.4 Histogram

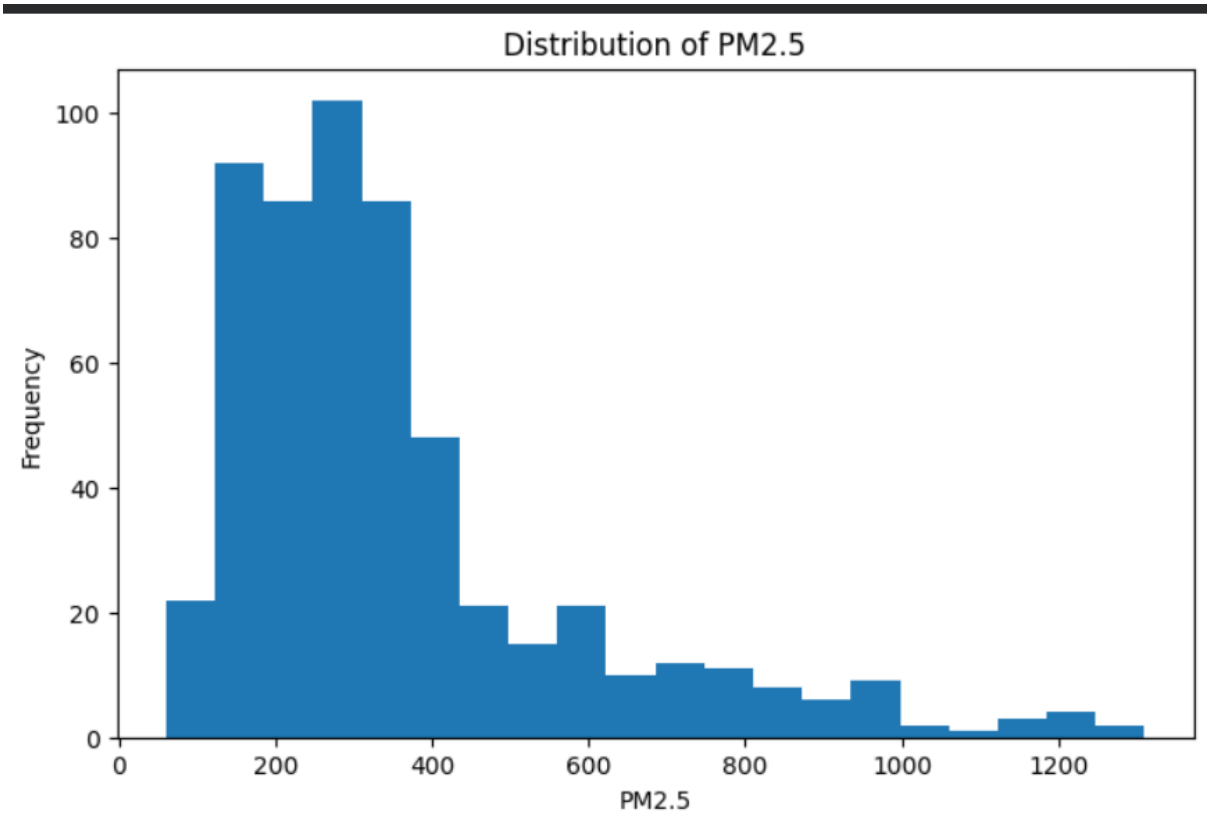
Description

A histogram is used to show the distribution of numerical data by grouping values into intervals (bins). It helps understand the frequency of data within different ranges.

Use Case

A histogram is useful for analyzing the distribution of exam scores, ages, heights, or any continuous

```
plt.figure(figsize=(8,5))

plt.hist(df["pm2_5"], bins=20)

plt.title("Distribution of PM2.5")
plt.xlabel("PM2.5")
plt.ylabel("Frequency")

plt.show()
```



2.2.5 Pie Chart

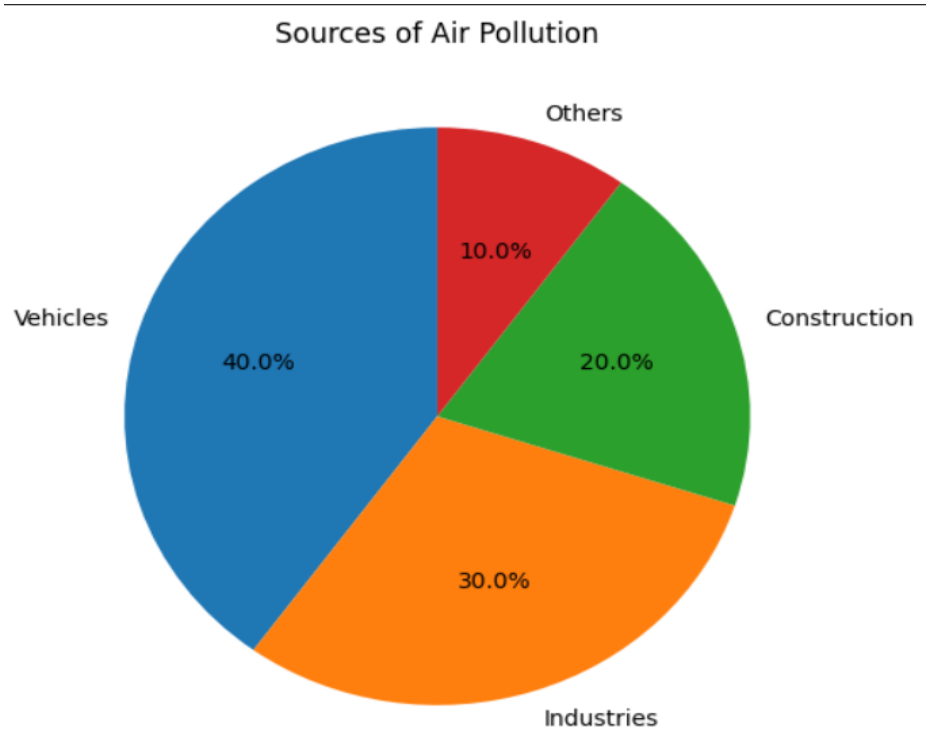
Description

A pie chart represents data as slices of a circle. Each slice shows the proportion of a category relative to the whole.

Use Case

A pie chart is useful for showing market share, budget allocation, or percentage distribution.

```
sources = ["Vehicles", "Industries", "Construction", "Others"]
values = [40,30,20,10]

plt.figure(figsize=(6,6))

plt.pie(values,
        labels=sources,
        autopct="%1.1f%%",
        startangle=90)

plt.title("Sources of Air Pollution")

plt.show()
```


Sources of Air Pollution
Others
10.0%
Vehicles
40.0%
Construction
20.0%
30.0%
Industries

2.2.6 Box Plot

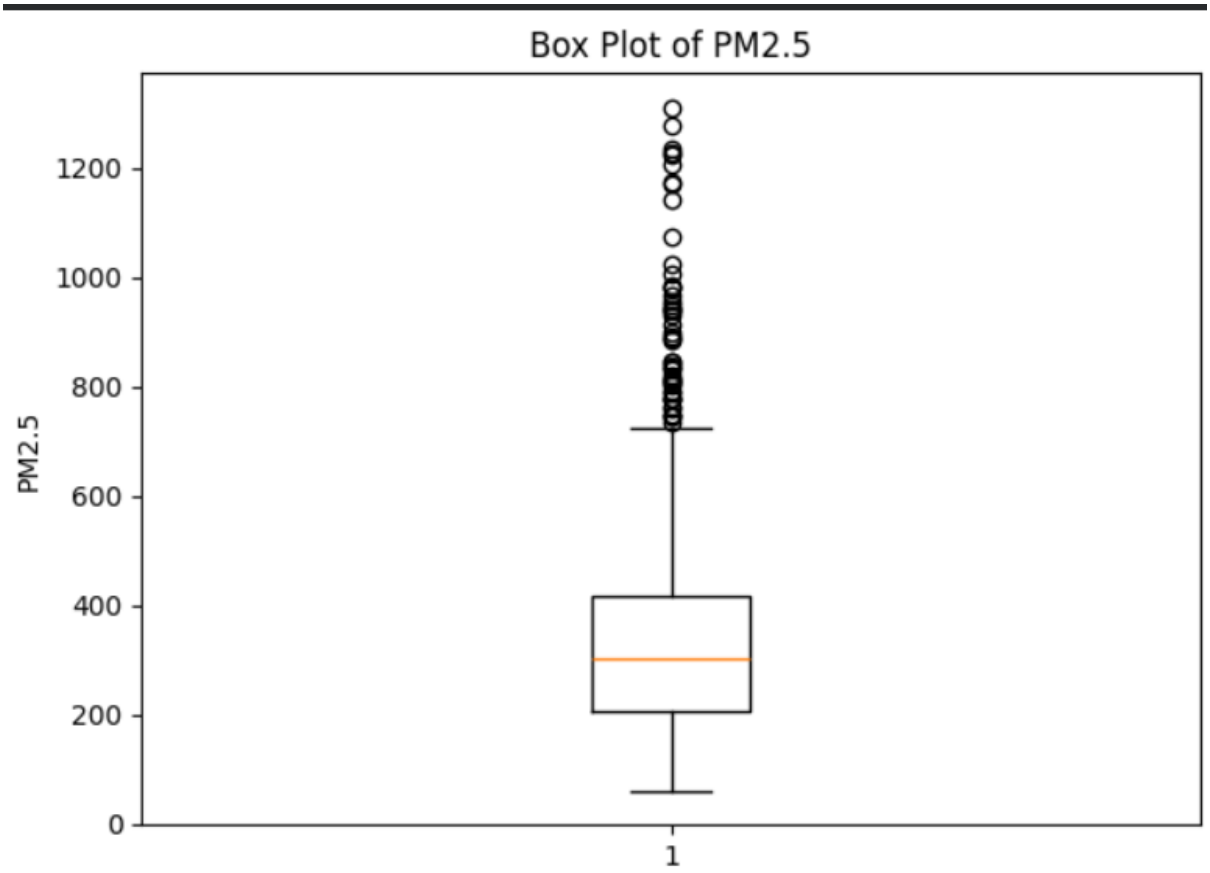
Description

A box plot summarizes the distribution of numerical data using the minimum quartile, median, third quartile, and maximum values. It also helps identify outliers .

Use Case

A box plot is useful for comparing data distribution

```
plt.figure(figsize=(7,5))

plt.boxplot(df["pm2_5"].dropna())

plt.title("Box Plot of PM2.5")
plt.ylabel("PM2.5")

plt.show()
```



3. Seaborn

3.1 Overview

Seaborn is an open-source Python library built on top of Matplotlib. It is designed to create attractive and informative statistical graphics with minimal code. Seaborn provides built-in themes, color palettes, and functions that simplify the visualization process. It integrates well with Pandas DataFrames, making it an excellent choice for exploratory data analysis and statistical visualization.

Features

- Easy to create attractive statistical graphs.
- Built on top of Matplotlib.
- Works seamlessly with Pandas DataFrames.
- Provides built-in themes and color palettes.
- Supports advanced visualizations such as heatmaps, pair plots, violin plots, and box plots.

Applications

- Air quality analysis
- Business analytics
- Scientific research
- Machine learning
- Exploratory Data Analysis (EDA)

3.2 Graph Types

3.2.1 Line Plot

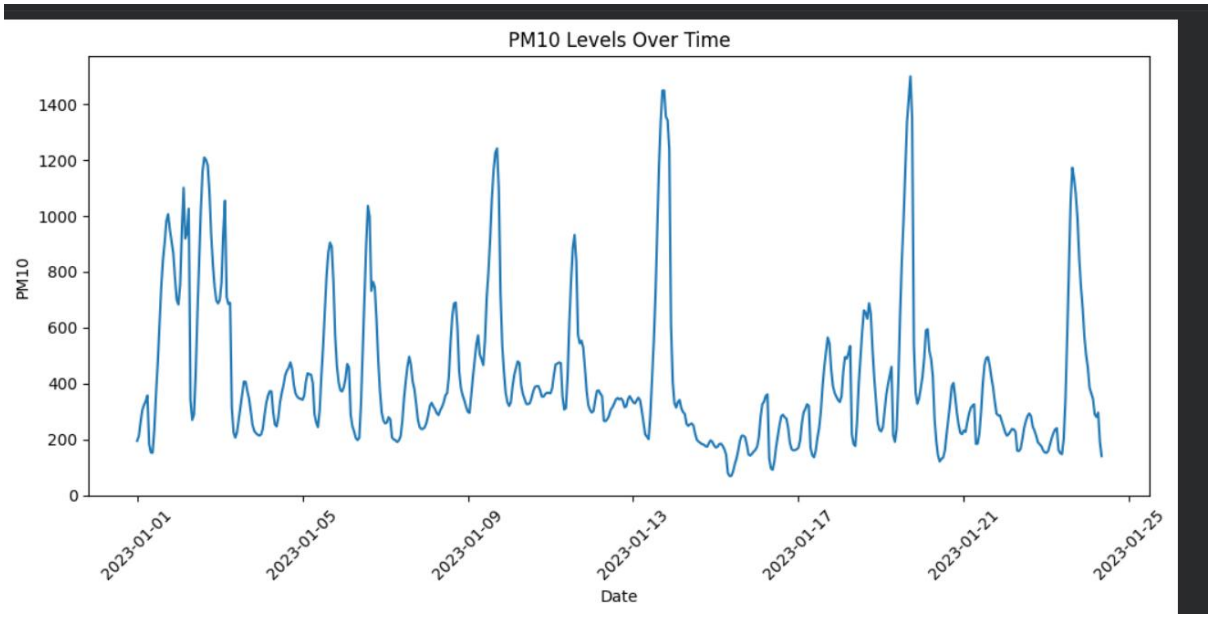
Description

A line plot displays changes in data over time by connecting data points with straight lines. It is useful for identifying trends and patterns in time-series data.

Use Case

To analyze how **PM10** pollution levels change over time in Delhi.

```
plt.figure(figsize=(12,5))

sns.lineplot(data=df, x="date", y="pm10")

plt.title("PM10 Levels Over Time")
plt.xlabel("Date")
plt.ylabel("PM10")

plt.xticks(rotation=45)

plt.show()
```


PM10 Levels Over Time
PM10
0
200
400
600
800
1000
1200
1400
2023-01-01
2023-01-05
2023-01-09
2023-01-13
2023-01-17
2023-01-21
2023-01-25
Date

3.2.2 Scatter Plot

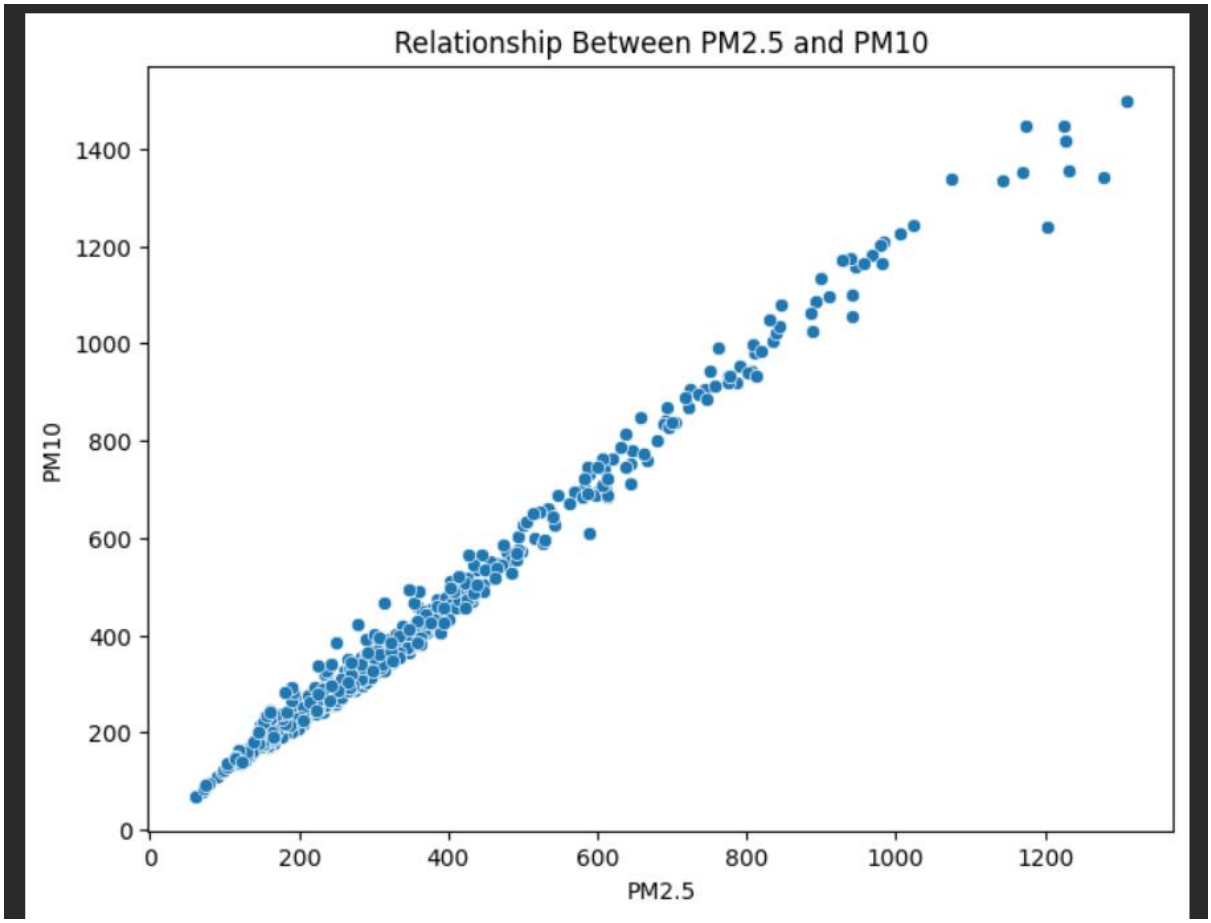
Description

A scatter plot displays the relationship between two numerical variables. Each point represents one observation.

Use Case

To study the relationship between **Nitrogen Dioxide (NO_2)** and **Ozone (O_3)**.

```
plt.figure(figsize=(8,6))

sns.scatterplot(data=df, x="pm2_5", y="pm10")

plt.title("Relationship Between PM2.5 and PM10")
plt.xlabel("PM2.5")
plt.ylabel("PM10")

plt.show()
```



3.2.3 Histogram

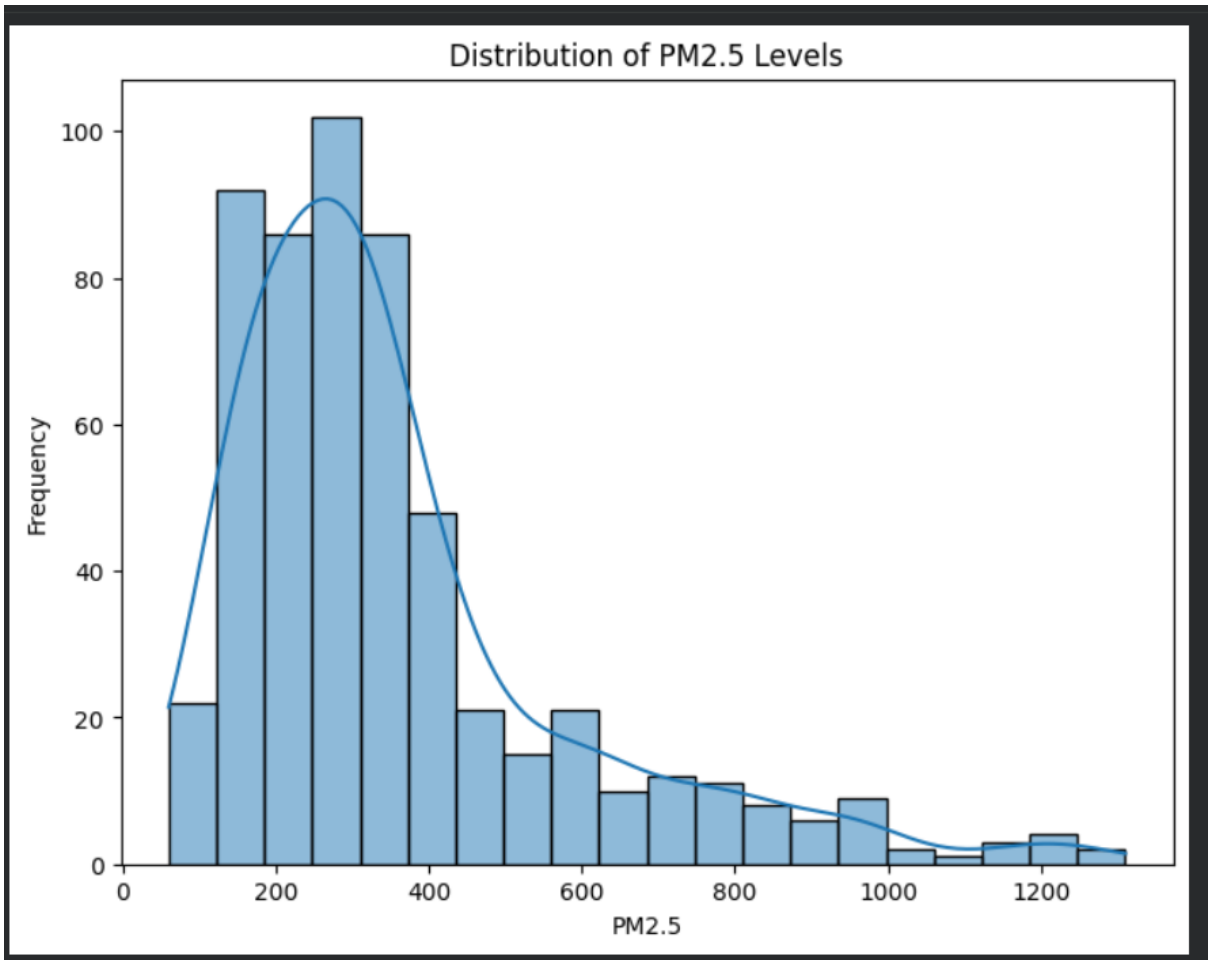
Description

A histogram shows how frequently values occur within different ranges.

Use Case

To examine the distribution of PM10 values.

```
plt.figure(figsize=(8,6))

sns.histplot(data=df, x="pm2_5", bins=20, kde=True)

plt.title("Distribution of PM2.5 Levels")
plt.xlabel("PM2.5")
plt.ylabel("Frequency")

plt.show()
```



3.2.4 Box Plot

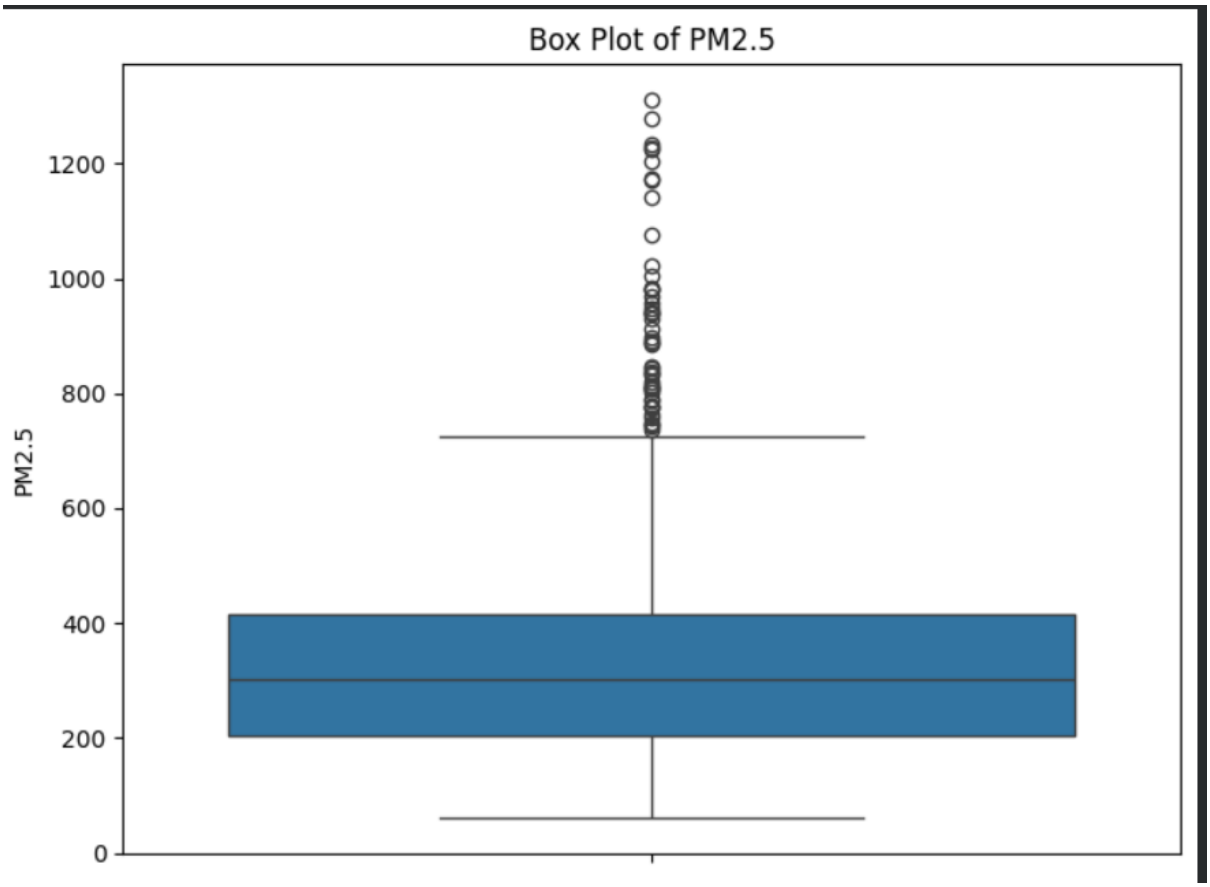
Description

A box plot summarizes data distribution and helps identify outliers.

Use Case

To detect unusually high or low PM10 values.

```
plt.figure(figsize=(8,6))

sns.boxplot(y=df["pm2_5"])

plt.title("Box Plot of PM2.5")
plt.ylabel("PM2.5")

plt.show()
```



3.2.5 Heatmap

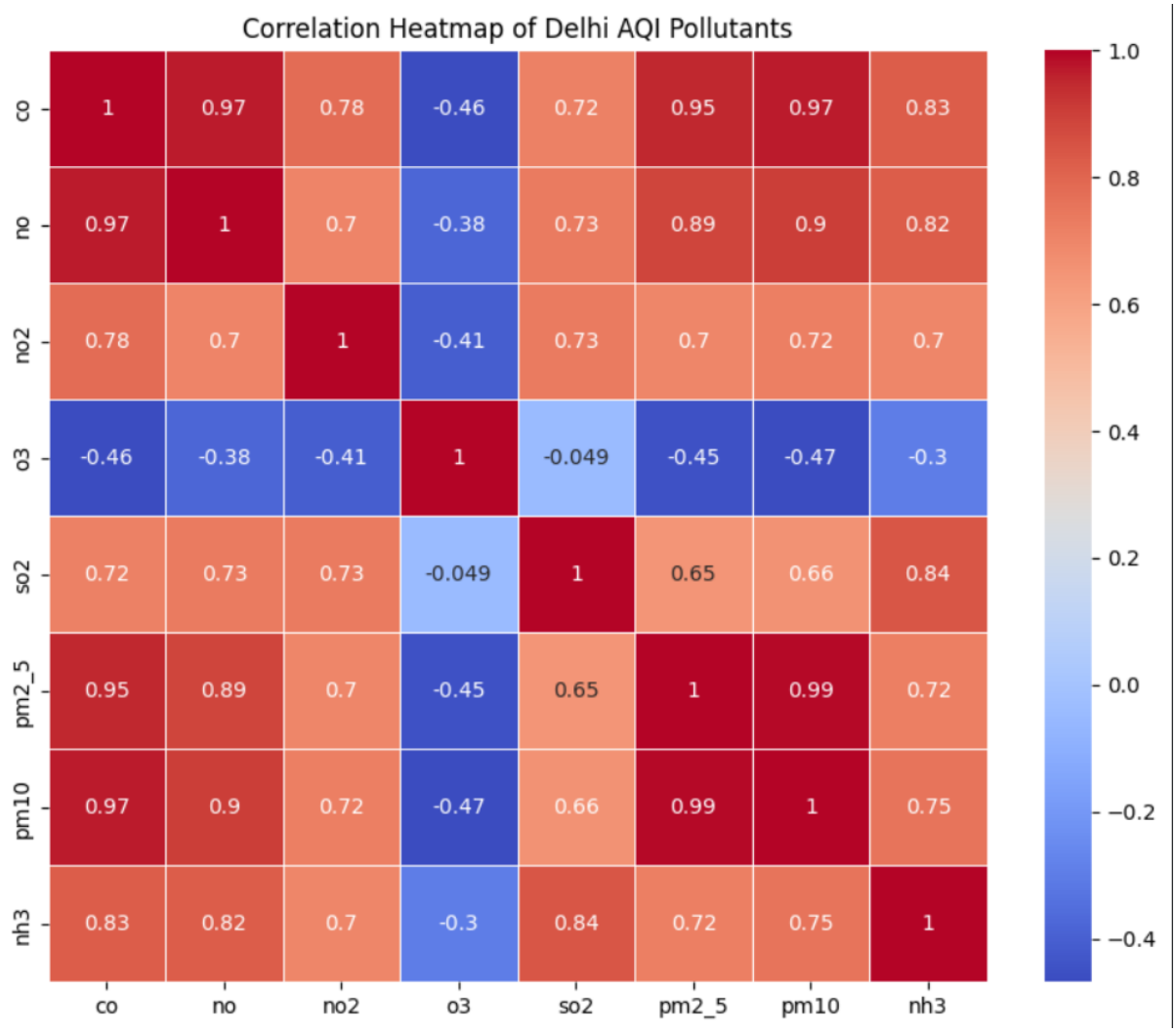
Description

A heatmap displays the correlation between multiple numerical variables using colors. Darker or brighter colors indicate stronger positive or negative relationships.

Use Case

To identify correlations among different pollutants in the Delhi AQI dataset.

```
plt.figure(figsize=(10,8))

correlation = df[["co","no","no2","o3","so2","pm2_5","pm10","nh3"]].corr()

sns.heatmap(correlation,
            annot=True,
            cmap="coolwarm",
            linewidths=0.5)

plt.title("Correlation Heatmap of Delhi AQI Pollutants")

plt.show()
```


Correlation Heatmap of Delhi AQI Pollutants
co
no
no2
o3
so2
pm2_5
pm10
nh3
1 0.97 0.78 -0.46 0.72 0.95 0.97 0.83
0.97 1 0.7 -0.38 0.73 0.89 0.9 0.82
0.78 0.7 1 -0.41 0.73 0.7 0.72 0.7
-0.46 -0.38 -0.41 1 -0.049 -0.45 -0.47 -0.3
0.72 0.73 0.73 -0.049 1 0.65 0.66 0.84
0.95 0.89 0.7 -0.45 0.65 1 0.99 0.72
0.97 0.9 0.72 -0.47 0.66 0.99 1 0.75
0.83 0.82 0.7 -0.3 0.84 0.72 0.75 1
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4

3.2.6 Pair Plot

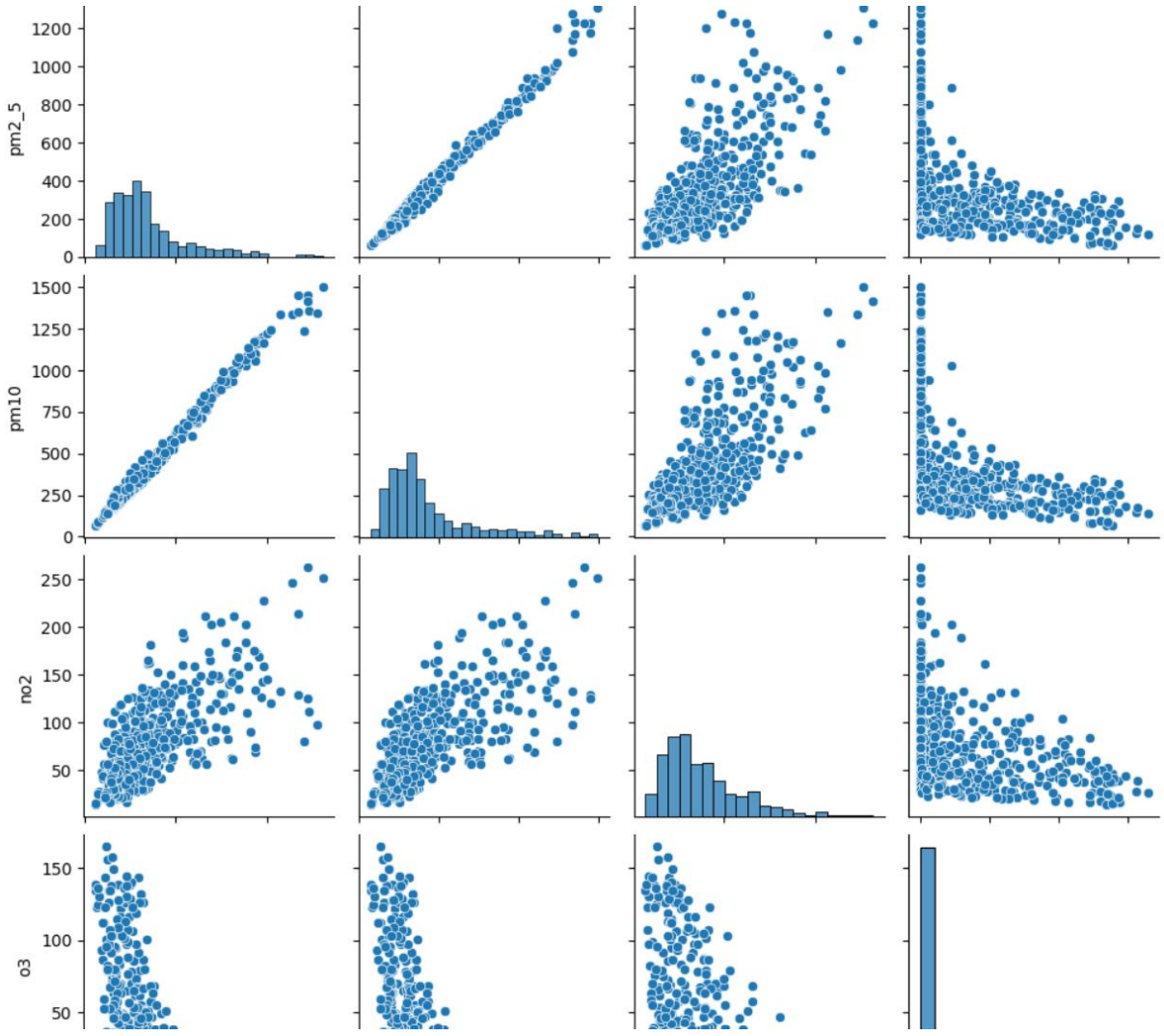
Description

A pair plot visualizes relationships between multiple numerical variables simultaneously using scatter plots and histograms.

Use Case

To compare relationships among major pollutants.

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.pairplot(df[["pm2_5", "pm10", "no2", "o3"]])

plt.show()
```



3.2.7 Violin Plot

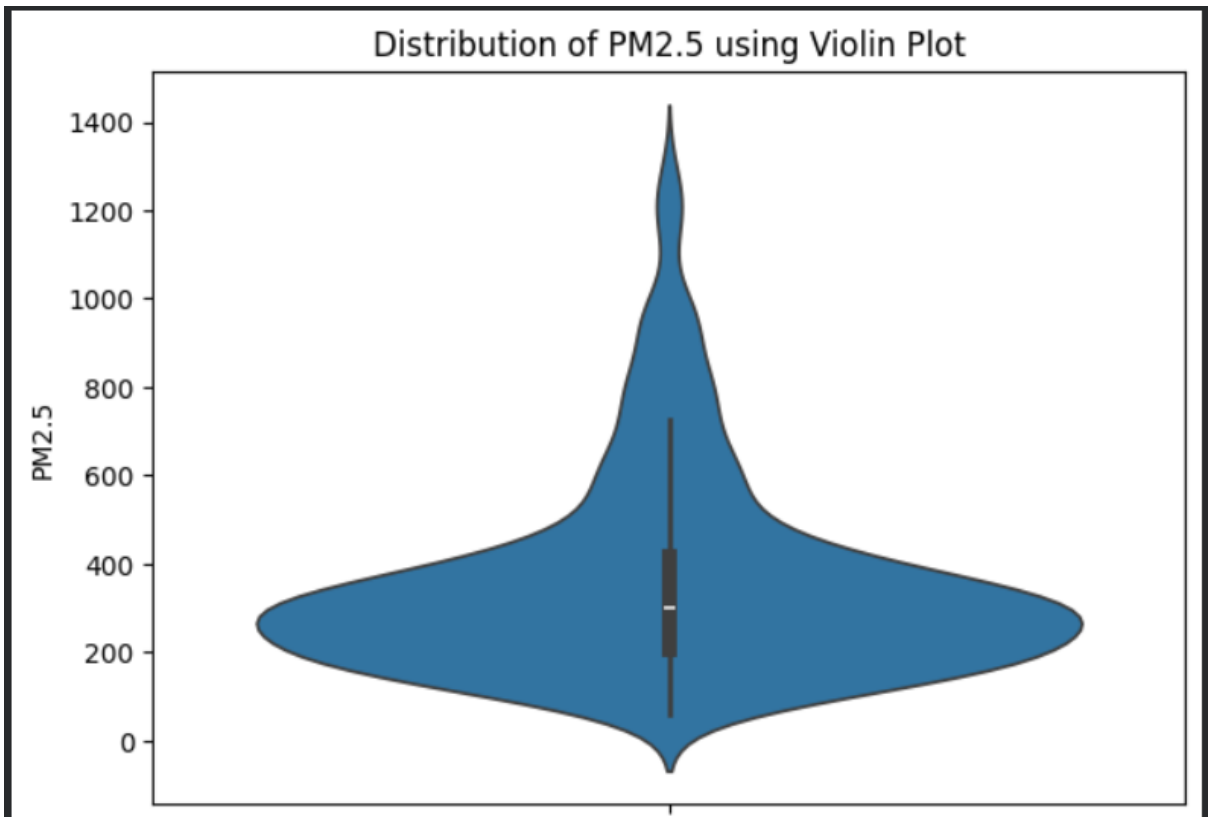
Description

A violin plot combines a box plot with a density plot, showing both the distribution and spread of data.

Use Case

To analyze the distribution of PM2.5 values.

```
plt.figure(figsize=(7,5))

sns.violinplot(y=df["pm2_5"])

plt.title("Distribution of PM2.5 using Violin Plot")
plt.ylabel("PM2.5")

plt.show()
```



3.2.8 Count Plot

Since your dataset has **continuous pollution values**, there is no category column to count directly. First, create pollution categories based on PM2.5 values.

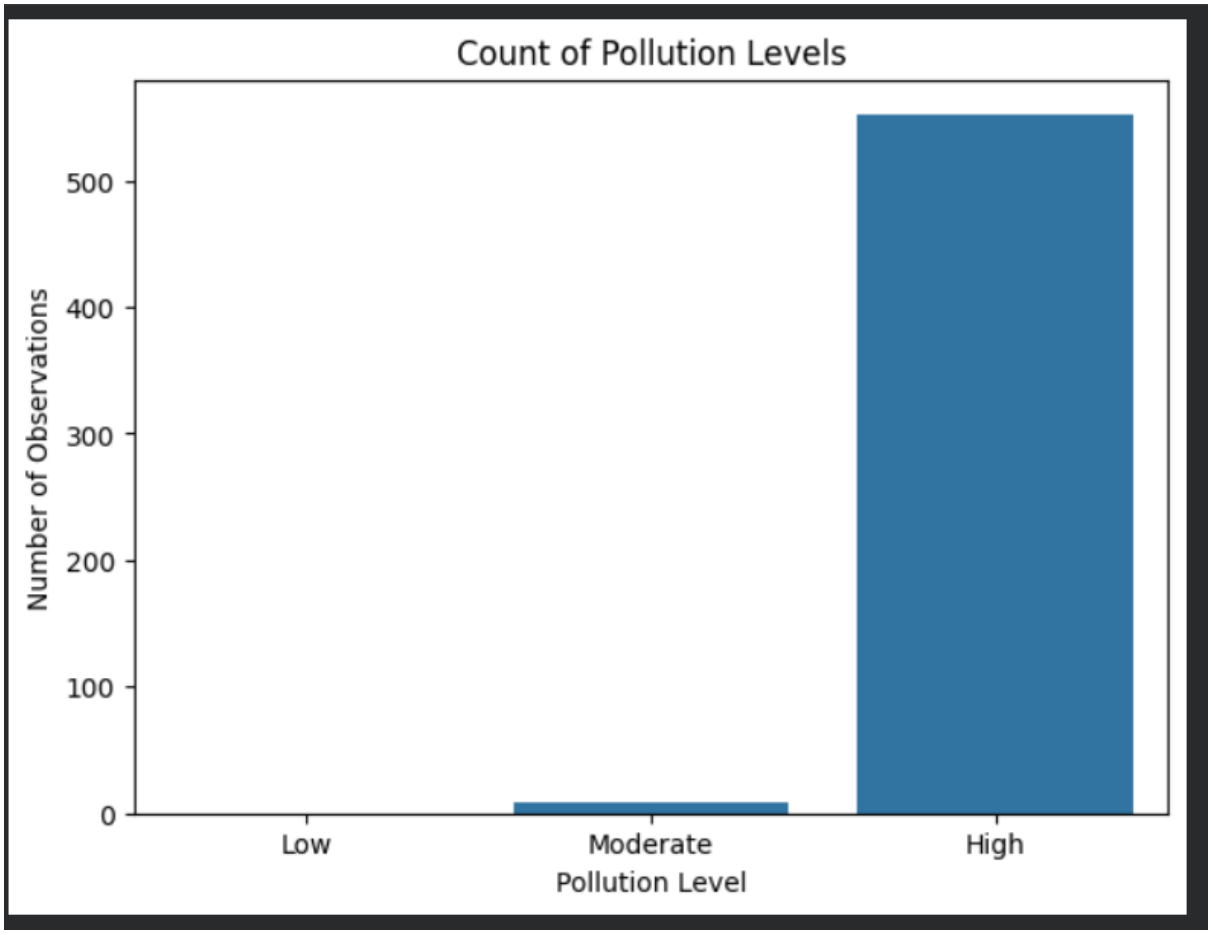
Description

A count plot shows the number of observations in each category.

Use Case

To count how many observations fall into Low, Moderate, and High pollution levels.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Create pollution categories based on PM2.5 values
df["Pollution_Level"] = pd.cut(
    df["pm2_5"],
    bins=[0, 50, 100, float("inf")],
    labels=["Low", "Moderate", "High"]
)

plt.figure(figsize=(7,5))

sns.countplot(data=df, x="Pollution_Level")

plt.title("Count of Pollution Levels")
plt.xlabel("Pollution Level")
plt.ylabel("Number of Observations")

plt.show()
```


Count of Pollution Levels
Number of Observations
0
100
200
300
400
500
Low
Moderate
High
Pollution Level

4:Comparison of Matplotlib and Seaborn

Feature	Matplotlib	Seaborn
Ease of Use	Moderate	Easy
Built On	Independent library	Built on Matplotlib
Customization	Very High	High
Statistical Graphs	Basic	Excellent
Appearance	Simple	Attractive
Learning Curve	Moderate	Easy
Best For	General-purpose plotting	Statistical data visualization
Integration	NumPy, Pandas	Pandas, Matplotlib

Comparison

Matplotlib

- Highly customizable.
- Supports many types of graphs.
- Suitable for detailed graph customization.
- Requires more code to create attractive plots.

Seaborn

- Built on top of Matplotlib.
- Produces visually appealing graphs with less code.
- Excellent for statistical analysis.
- Works seamlessly with Pandas DataFrames.

5: Conclusion

Conclusion

This project demonstrated the use of two popular Python visualization libraries, **Matplotlib** and **Seaborn**, using the Delhi AQI dataset. Various graphs such as line plots, scatter plots, histograms, box plots, heatmaps, pair plots, violin plots, and count plots were created to analyze air pollution data. Matplotlib provides flexibility and customization for creating different types of charts, while Seaborn simplifies statistical visualization with attractive and informative plots. The visualizations helped identify trends, relationships, distributions, and correlations among different pollutants. Overall, both libraries are powerful tools for data analysis and play an important role in understanding real-world datasets through effective visualization.

6: References

1. Matplotlib Documentation – https://matplotlib.org/
2. Seaborn Documentation – https://seaborn.pydata.org/
3. Pandas Documentation – https://pandas.pydata.org/